UNIVERSITE DE FIANARANTSOA
ECOLE NATIONALE D'INFORMATIQUE
RAPPORT DE PROJET INTERNET DES OBJETS
Mention : Informatique
Parcours : Administration Systeme et Reseau - Master 2

Conception et realisation d'un systeme IoT de surveillance des inondations : FloodGuard Smart Area

Theme choisi : Detection des inondations par capteurs IoT et visualisation temps reel.

Prototype : ESP32 simule sous Wokwi, communication MQTT, pile Docker locale Telegraf - InfluxDB - Grafana.

Presente par :

TAHINTSOA Mica Tahintsoa

Annee Universitaire 2025 - 2026

SOMMAIRE

Introduction	
Chapitre 1 : Presentation du projet	Problematique, objectifs, besoins fonctionnels, roles
Chapitre 2 : Conception et realisation	Architecture, materiels, protocoles, stockage, plateforme IoT, prototype
Chapitre 3 : Integration de la securite et scenarios de tests	Risques, mesures de securite, validation fonctionnelle
Conclusion	
References	
Annexes	Payload MQTT, commandes et requetes utiles

LISTE DES FIGURES

Figure 1	Architecture generale de FloodGuard Smart Area
Figure 2	Prototype Wokwi avec ESP32, capteurs et actionneurs
Figure 3	Processus de calcul et de publication des donnees
Figure 4	Tableau de bord Grafana - vue generale
Figure 5	Interaction avec le capteur ultrason dans Wokwi
Figure 6	Visualisation des etats, tendance et buzzer
Figure 7	Exemple de journal serie et payload JSON MQTT

LISTE DES TABLEAUX

Table 1	Materiels et equivalences entre simulation et prototype reel
Table 2	Besoins fonctionnels du systeme
Table 3	Flux de donnees et metriques transmises
Table 4	Panels Grafana et interpretation
Table 5	Scenarios de tests et resultats attendus

LISTE DES ABREVIATIONS

Abreviation	Signification
IoT / IdO	Internet of Things / Internet des Objets
ESP32	Microcontroleur avec Wi-Fi utilise pour l’acquisition et la publication des mesures
MQTT	Protocole leger de messagerie publish/subscribe
HC-SR04	Capteur ultrason utilise pour mesurer une distance
JSON	Format de donnees structurees envoye par MQTT
RSSI	Received Signal Strength Indicator, indicateur de puissance Wi-Fi
SAST/DAST	Tests de securite applicative statique/dynamique, utiles dans une demarche DevSecOps

INTRODUCTION

L'Internet des objets permet d'interconnecter des capteurs, des actionneurs et des plateformes logicielles afin de surveiller un environnement physique en temps reel. Dans ce projet, le theme retenu est la detection des inondations, un cas concret ou les variations du niveau d'eau et l'intensite de pluie doivent etre observees rapidement pour anticiper les risques.

Le systeme propose, nomme FloodGuard Smart Area, simule une zone basse sujette aux inondations frequentes. Le prototype repose sur un ESP32 simule avec Wokwi, un capteur ultrason pour estimer le niveau d'eau, un potentiometre pour representer la pluie, des LEDs et un buzzer pour l'alerte locale. Les mesures sont publiees en MQTT vers un broker public puis consommees par une pile locale conteneurisee : Telegraf, InfluxDB et Grafana.

Ce rapport presente le projet, son architecture, son fonctionnement, les panels Grafana, les mesures de securite et les scenarios de validation.

Chapitre 1 : Presentation du projet

1.1. Theme choisi

Le theme choisi est la detection des inondations. Le prototype vise a surveiller une zone a risque comme un quartier bas, un canal d'evacuation ou un bassin de retention. L'objectif est de detecter la montee de l'eau, d'estimer le niveau de risque et de fournir une visualisation claire aux utilisateurs.

1.2. Problematique

Dans une zone frequemment exposee aux fortes pluies, une inondation peut evoluer rapidement. Une surveillance manuelle est lente, dependante de la presence humaine et ne permet pas toujours d'emettre une alerte au bon moment. Le probleme central est donc de concevoir un systeme capable de mesurer, transmettre, analyser et afficher l'etat d'une zone a risque en temps quasi reel.

1.3. Objectifs

- Mesurer une distance entre un capteur ultrason et la surface de l'eau afin de deduire un niveau d'eau.
- Simuler l'intensite de pluie et l'integrer dans le calcul du risque.
- Envoyer les mesures sous forme de payload JSON via MQTT.
- Stocker les donnees temporelles dans InfluxDB.
- Afficher un tableau de bord Grafana compréhensible pour la surveillance et la demonstration.
- Declencher une alerte locale par LED et buzzer selon le niveau de risque.

1.4. Acteurs et besoins fonctionnels

Les acteurs sont l'operateur de surveillance, le systeme IoT et l'administrateur technique. L'operateur consulte les mesures et les alertes. Le systeme IoT collecte les donnees et publie la telemetrie. L'administrateur maintient la pile Docker, Grafana et les configurations MQTT/InfluxDB.

Acteur	Besoins fonctionnels
Operateur	Consulter le niveau d’eau, la pluie, le niveau de risque et les alertes sur Grafana.
Systeme IoT	Mesurer, calculer le risque, publier les donnees MQTT, activer LEDs et buzzer.
Administrateur	Configurer MQTT, Telegraf, InfluxDB, Grafana et maintenir les conteneurs Docker.

Table 1 - Besoins fonctionnels principaux du systeme.

1.5. Roles dans le projet

Le travail de conception, de simulation, d'integration MQTT/Grafana et de documentation est assure par TAHINTSOA Mica Tahintsoa. Si le projet est realise en groupe, les autres noms et responsabilites peuvent etre ajoutes dans cette section : cablage, tests, documentation, securite, presentation.

Chapitre 2 : Conception et realisation

2.1. Architecture generale

L'architecture retenue separe clairement l'acquisition, la transmission, le stockage et la visualisation. L'ESP32 joue le role de noeud IoT. Le broker MQTT assure la communication decouplee entre le noeud simule et la plateforme locale. Telegraf lit les messages MQTT et les transforme en mesures pour InfluxDB. Grafana interroge InfluxDB pour afficher l'etat du systeme.

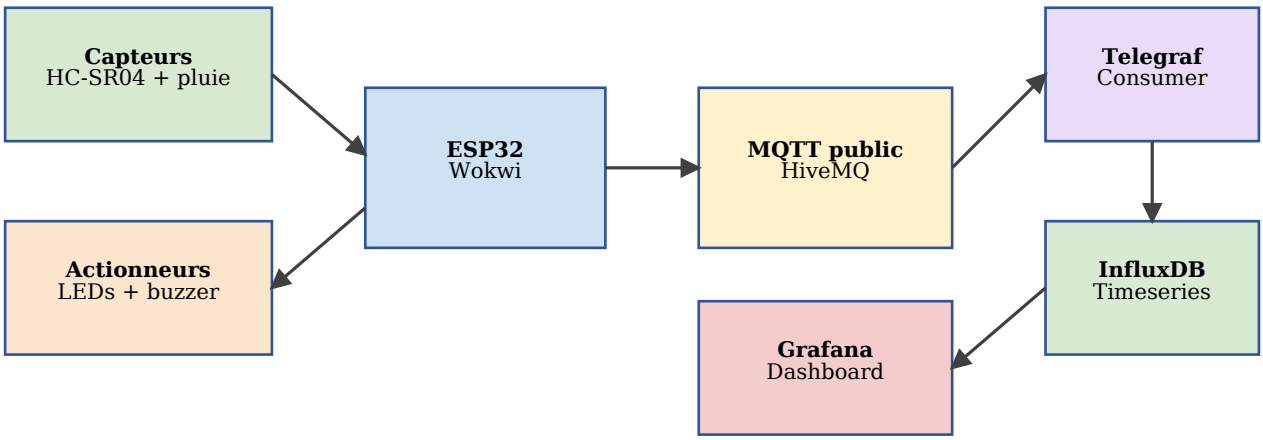

Figure 1 - Architecture generale de FloodGuard Smart Area.

2.2. Choix des materiels et equivalences de simulation

Element	Role dans la simulation	Equivalent reel apres financement
ESP32	Noeud IoT avec Wi-Fi et logique embarquee	ESP32 DevKit reel dans boitier protege
HC-SR04	Mesure de distance capteur-eau	JSN-SR04T etanche, plus adapte au terrain
Potentiometre	Simulation de l'intensite de pluie	Module capteur de pluie analogique
LED verte/jaune/rouge	Signalisation locale des etats	Voyants 5 mm ou balise lumineuse
Buzzer	Alerte sonore en cas de risque	Buzzer actif ou sirene via transistor/MOSFET/relais

Table 2 - Materiels utilises et correspondance avec une realisation physique.

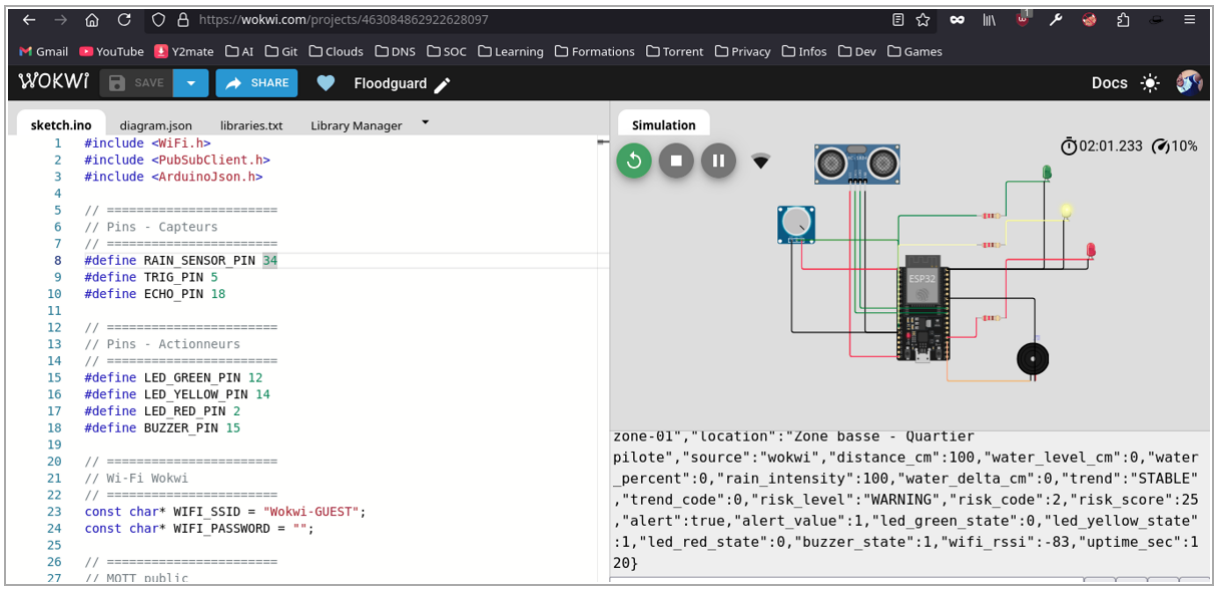

Figure 2 - Prototype Wokwi avec ESP32, HC-SR04, potentiometre, LEDs et buzzer.

2.3. Connectivite et protocoles

Le protocole choisi est MQTT, car il est leger, adapte aux objets connectes et fonctionne avec un modele publish/subscribe. L'ESP32 publie les donnees sur le topic `eni/m2/iot/floodguard/mica-tahintsoa-2026/telemetry`. Le broker utilise pendant la demonstration est `broker.hivemq.com` sur le port 1883. Ce choix permet de faire fonctionner Wokwi sans exposer directement le PC local.

Dans la version de production, le broker public devra etre remplace par un broker MQTT prive, authentifie et chiffre par TLS.

2.4. Fonctionnement embarque du prototype

Le capteur ultrason mesure une distance en centimetres. Le code considere une hauteur maximale de reference de 100 cm. Le niveau d'eau est donc calcule par la formule :

```
water_level_cm = MAX_WATER_HEIGHT_CM - distance_cm
water_percent  = (water_level_cm / MAX_WATER_HEIGHT_CM) * 100
```

Ainsi, si la distance mesuree est 100 cm, le niveau d'eau est 0 %. Si la distance descend a 20 cm, le niveau d'eau calcule devient 80 %. Le potentiometre simule l'intensite de pluie sur une echelle de 0 a 100 %. Ces deux informations sont ensuite combinees pour calculer le score de risque.

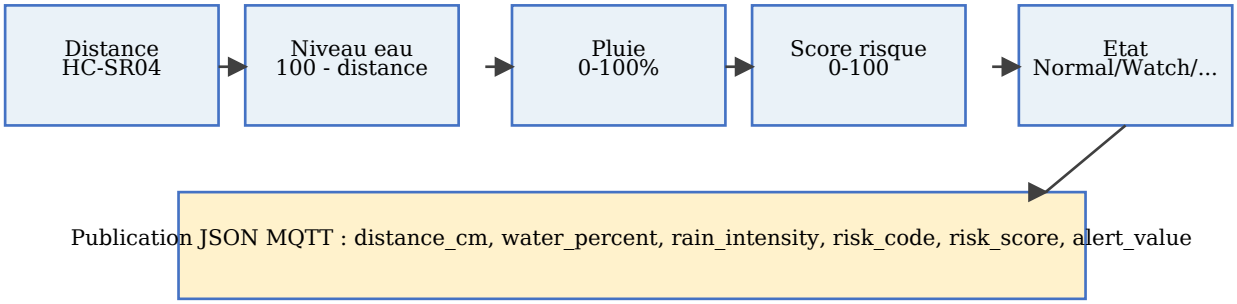

Figure 3 - Processus de calcul des indicateurs publies par MQTT.

2.5. Modele de donnees MQTT

Chaque publication MQTT contient un document JSON. Les champs numeriques sont stockes comme metriques dans InfluxDB et les champs textuels comme `project`, `device_id`, `location`, `risk_level`, `trend` et `source` sont utilises comme tags par Telegraf.

Champ	Type	Role
distance_cm	nombre	Distance mesuree par le capteur ultrason.
water_level_cm	nombre	Hauteur d’eau deduite de la distance.
water_percent	nombre	Niveau d’eau normalise entre 0 et 100 %.
rain_intensity	nombre	Intensite de pluie simulee.
water_delta_cm	nombre	Variation du niveau d’eau entre deux mesures.
risk_code	nombre	0=NORMAL, 1=WATCH, 2=WARNING, 3=CRITICAL.
risk_score	nombre	Score synthetique du danger entre 0 et 100.
alert_value	nombre	0 aucune alerte, 1 alerte active.
buzzer_state	nombre	Etat de la sirene locale.

Table 3 - Principales metriques envoyees par le noeud IoT.

2.6. Pile logicielle locale

La plateforme de reception est executee en conteneurs Docker. Elle contient InfluxDB pour le stockage temporel, Telegraf pour la consommation MQTT, Grafana pour les tableaux de bord, Node-RED comme plateforme de flux optionnelle et un conteneur `mqtt-listener` pour le debug MQTT.

Service	Role	Port
influxdb	Stockage time-series des metriques IoT	8086
telegraf	Lecture MQTT et ecriture dans InfluxDB	interne
grafana	Visualisation et supervision	3000
node-red	Flux et debug IoT optionnel	1880
mqtt-listener	Verification des messages MQTT dans les logs Docker	interne

2.7. Tableau de bord Grafana et explication des panels

Le dashboard Grafana est l'interface utilisateur principale. Il doit permettre de comprendre l'etat du terrain sans lire les logs. Les graphes initiaux pouvaient etre confus car les tags dynamiques comme `risk_level` et `trend` creaient plusieurs series dans certains panels. Pour une lecture claire, les panels de synthese doivent toujours afficher la derniere valeur utile et les courbes doivent etre groupees par metrique.

Figure 4 - Vue generale du tableau de bord Grafana.

Panel	Metrique	Interpretation
Etat du risque	risk_code	Affiche l'etat courant : NORMAL, WATCH, WARNING ou CRITICAL. C'est l'indicateur le plus important pour l'operateur.
Niveau d'eau calcule	water_percent	Pourcentage de remplissage deduit de la distance ultrason. 0 % signifie eau basse, 85 % ou plus indique un risque critique.
Intensite de pluie	rain_intensity	Montre la pluie simulee. Une pluie forte peut produire WATCH ou WARNING meme si l'eau n'est pas encore montee.
Alerte active	alert_value	Indique si une alerte est declenchee. 1 = alerte, 0 = aucune.
Hauteur d'eau	water_level_cm	Valeur physique deduite en cm. Elle rend le pourcentage plus comprehensible.
Distance capteur-eau	distance_cm	Plus cette distance baisse, plus l'eau monte. Utile pour diagnostiquer le capteur.
Score de risque	risk_score	Synthese calculee de l'eau, de la pluie et de la tendance.
Sirene / buzzer	buzzer_state	Etat de l'alerte sonore locale.
Niveau + pluie	water_percent/rain_intensity	Compare les deux facteurs principaux dans le temps.
Timeline des etats	risk_code	Visualise les transitions entre NORMAL, WATCH, WARNING et CRITICAL.
Variation niveau eau	water_delta_cm	Montree si l'eau monte, descend ou reste stable.

Table 4 - Panels Grafana et interpretation fonctionnelle.

2.8. Diagnostic du panel "Niveau d'eau (%)"

Le panel `Niveau d'eau (%)` n'etait pas forcement en erreur. Les logs montrent que le systeme publiait bien `water_percent`, mais avec une valeur egale a 0 parce que `distance_cm` restait a 100. D'apres la formule, cela signifie que l'eau simulee est au niveau le plus bas.

```
{"distance_cm":100,"water_level_cm":0,"water_percent":0,"risk_level":"NORMAL"}
```

Pour tester ce graphe, il faut cliquer sur le HC-SR04 dans Wokwi et reduire la distance de l'objet simule. Par exemple, a 30 cm, le niveau d'eau calcule devient 70 %. Le graphe commence alors a afficher une valeur significative.

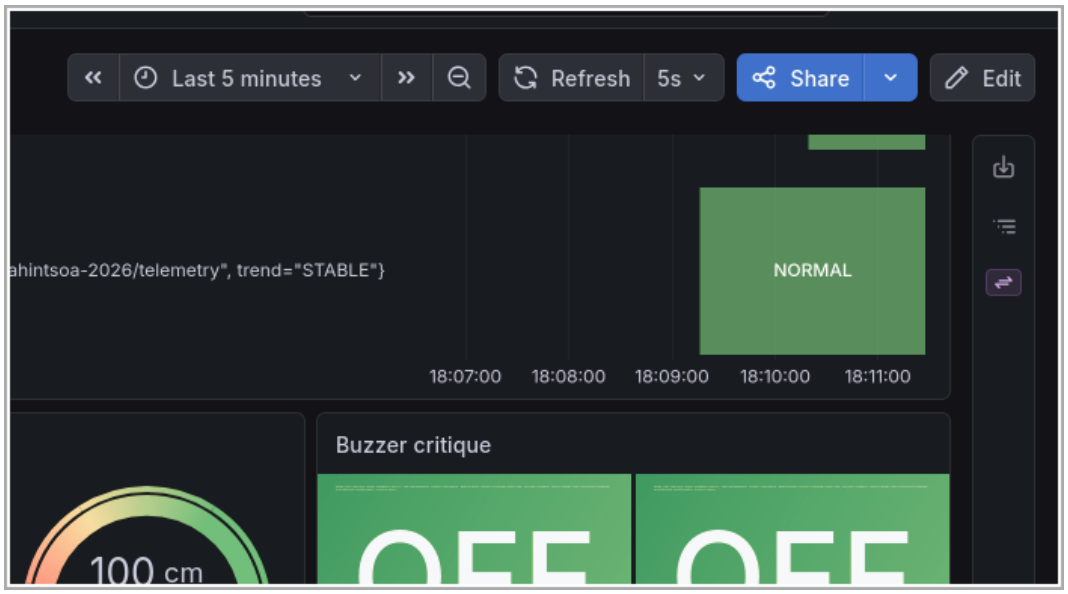

Figure 5 - Reglage de la distance du capteur ultrason dans Wokwi.

2.9. Recommandations Grafana pour eviter les graphes multiples

Lorsque `risk_level` et `trend` sont definis comme tags, InfluxDB cree une serie differente pour chaque combinaison de tags. Cela explique pourquoi certaines captures montrent plusieurs jauges pour un meme panel. Deux corrections sont possibles :

- Dans les requetes Flux des panels de synthese, ajouter `group(columns: [])` avant `last()` pour fusionner les series avant d'afficher la derniere valeur globale.
- Ou bien retirer les tags trop dynamiques (`risk_level`, `trend`) de `tag_keys` et les conserver seulement comme champs si l'on veut eviter la fragmentation des series.

La solution la plus simple pour une soutenance est d'utiliser `group(columns: [])` dans les panels de type stat/gauge afin d'obtenir une seule valeur claire.

Figure 6 - Visualisation des etats, tendance, distance capteur-eau et buzzer.

Chapitre 3 : Integration de la securite et scenarios de tests

3.1. Securite de la communication MQTT

La demonstration utilise `broker.hivemq.com` sur le port 1883. Ce choix est pratique pour une simulation Wokwi, mais il n'est pas adapte a la production : le topic est public, les messages ne sont pas chiffres et n'importe quel client connaissant le topic peut publier de fausses donnees.

En production, il faut utiliser un broker MQTT prive avec authentification, TLS sur le port 8883, ACL par topic et rotation des identifiants. Les devices devraient publier uniquement dans leur propre topic, par exemple `floodguard/devices/esp32-zone-01/telemetry`.

3.2. Securite de la plateforme Docker

- Ne pas exposer InfluxDB directement sur Internet.
- Proteger Grafana par mot de passe fort, HTTPS et reverse proxy.
- Ne pas versionner le fichier `.env` contenant les tokens et mots de passe.
- Scanner les images Docker avec Trivy ou Docker Scout.
- Limiter les droits des conteneurs et utiliser le principe du moindre privilege.

3.3. Securite materielle pour la future realisation

Le HC-SR04 de simulation devra etre remplace par un capteur ultrason etanche. Si le signal ECHO sort en 5 V, il faudra un diviseur de tension ou un level shifter pour proteger les entrees 3.3 V de l'ESP32. Une sirene reelle ne doit jamais etre alimentee directement par un GPIO ; elle doit passer par un transistor, un MOSFET ou un relais. L'ensemble devra etre place dans un boitier IP65 minimum.

3.4. Scenarios de tests

Scenario	Manipulation	Resultat attendu
Situation normale	Distance HC-SR04 elevee, pluie faible	LED verte allumee, risk_level=NORMAL, alert_value=0.
Pluie forte	Augmenter le potentiometre	LED jaune, etat WATCH ou WARNING selon le seuil.
Montee d’eau	Reduire la distance HC-SR04	water_percent augmente, risk_score augmente.
Risque critique	Distance faible ou eau >= 85 %	LED rouge, buzzer actif, risk_level=CRITICAL.
Perte/instabilite MQTT	Observer les logs avec rc=-4 ponctuel	Le client tente une reconnexion et reprend la publication.

Table 5 - Scenarios de tests fonctionnels et resultats attendus.


```
zone-01","location":"Zone basse - Quartier
pilote","source":"wokwi","distance_cm":100,"water_level_cm":0,"water
_percent":0,"rain_intensity":1,"water_delta_cm":-8.250931,"trend":"F
ALLING","trend_code":-1,"risk_level":"NORMAL","risk_code":0,"risk_sc
ore":0,"alert":false,"alert_value":0,"led_green_state":1,"led_yellow
_state":0,"led_red_state":0,"buzzer_state":0,"wifi_rssi":-91,"uptime
_sec":106}
```


Figure 7 - Exemple de logs Wokwi et de payload JSON publie par MQTT.

3.5. Analyse des resultats obtenus

Les logs confirment que la connexion Wi-Fi Wokwi fonctionne, que la resolution DNS du broker MQTT est correcte et que les publications MQTT sont acceptees. Les payloads envoyes contiennent les metriques necessaires a la supervision : distance, niveau d'eau, pluie, tendance, risque, alerte, buzzer et RSSI Wi-Fi.

Les captures Grafana montrent que les etats NORMAL, WATCH et WARNING sont visibles. Elles montrent egalement un comportement important : si la distance du capteur reste a 100 cm, le niveau d'eau calcule reste a 0 %. Ce n'est pas un bug du calcul ; c'est une situation de test qui correspond a une eau tres basse. La validation du panel niveau d'eau se fait donc en changeant la distance du HC-SR04 dans Wokwi.

L'autre probleme observe est la multiplication de series dans certains panels. Cette multiplication provient du fait que les tags dynamiques `risk_level` et `trend` changent au fil du temps. Pour un dashboard plus propre, les panels de valeur actuelle doivent regrouper les series avec `group(columns: [])` et utiliser `last()`.

CONCLUSION

FloodGuard Smart Area demontre qu'un systeme IoT de detection d'inondation peut etre concu et valide sans disposer immediatement de tout le materiel physique. La simulation Wokwi permet de representer le comportement des capteurs et actionneurs, tandis que MQTT, Telegraf, InfluxDB et Grafana fournissent une chaine complete de collecte, stockage et visualisation.

Le projet repond aux exigences d'un systeme IoT complet : materiel simule, connectivite, logiciel d'analyse, stockage et interface utilisateur. Les principales limites concernent le broker MQTT public et la simulation d'un capteur de pluie par potentiometre. Pour une version financee, l'architecture pourra etre deployee avec un ESP32 reel, un capteur ultrason etanche, un capteur pluie, un broker MQTT prive securise et un boitier terrain protege.

La partie Grafana joue un role central : elle transforme les mesures brutes en indicateurs exploitables pour un operateur. Les panels les plus importants sont l'etat de risque, le niveau d'eau calcule, l'intensite de pluie, l'alerte active, le score de risque et la timeline des etats.

REFERENCES

Documentation Wokwi - ESP32 Wi-Fi et simulation de composants : https://docs.wokwi.com/

HiveMQ Public MQTT Broker : https://www.hivemq.com/mqtt/public-mqtt-broker/

InfluxDB Docker et Flux : https://docs.influxdata.com/

Telegraf MQTT Consumer Input Plugin : https://docs.influxdata.com/telegraf/

Grafana InfluxDB datasource et dashboards : https://grafana.com/docs/

Eclipse Mosquitto clients : https://mosquitto.org/

ANNEXES

Annexe A - Exemple de payload MQTT


```
{
  "project": "FloodGuard Smart Area",
  "device_id": "esp32-floodguard-zone-01",
  "location": "Zone basse - Quartier pilote",
  "source": "wokwi",
  "distance_cm": 100,
  "water_level_cm": 0,
  "water_percent": 0,
  "rain_intensity": 96,
  "risk_level": "WARNING",
  "risk_code": 2,
  "risk_score": 24,
  "alert_value": 1,
  "buzzer_state": 1
}
```


Annexe B - Commandes de verification

```
docker compose up -d

docker compose logs -f mqtt-listener

docker compose exec influxdb influx query '
from(bucket:"floodguard")
  |> range(start:-10m)
  |> filter(fn: (r) => r._measurement == "floodguard")
  |> limit(n:20)
' --org "$INFLUXDB_ORG" --token "$INFLUXDB_TOKEN"
```

Annexe C - Requete Flux recommandee pour une jauge unique

```
from(bucket: "floodguard")
  |> range(start: -30m)
  |> filter(fn: (r) => r._measurement == "floodguard")
  |> filter(fn: (r) => r._field == "water_percent")
  |> group(columns: [])
  |> last()
```

Cette requete evite que Grafana affiche plusieurs jauges pour une seule mesure lorsque certains tags changent dans le temps.